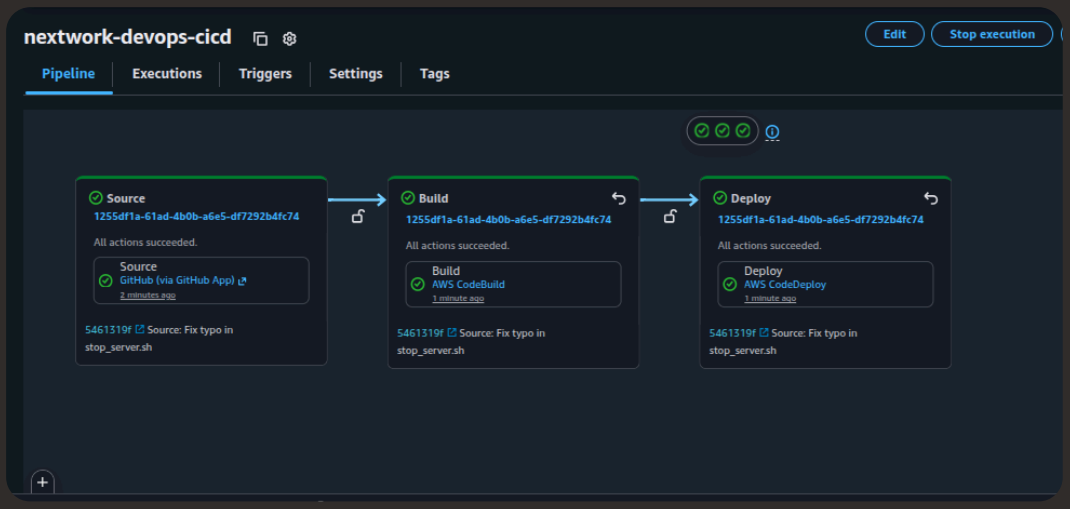
BUILD A CI/CD PIPELINE WITH AWS

By Precious Awoyemi | September 2026



Introducing Today's Project!

In this project, I will demonstrate how to use CodePipeline to set up a CI/CD pipeline. CodePipeline will automate the flow from GitHub all the way to CodeDeploy. By the end of this project, I'll be able to simply push a change to the code and see the updated web app live in production, without having to manually go into CodeBuild or CodeDeploy myself.

Key tools and concepts

The services I used included CodePipeline, CodeDeploy, CodeBuild, CodeArtifact, GitHub, VS Code, EC2, S3, IAM, and CloudFormation. Key concepts I learned include the different stages of a CI/CD pipeline, handling rollbacks, and working with webhooks.

Project reflection

This project took just under 2.5 hours, including documentation and troubleshooting time. The most challenging part was dealing with permission errors during the rollback (Amazon Q saved the day). The most rewarding part was seeing the live, deployed web app update automatically, without me needing to rebuild and redeploy the project manually

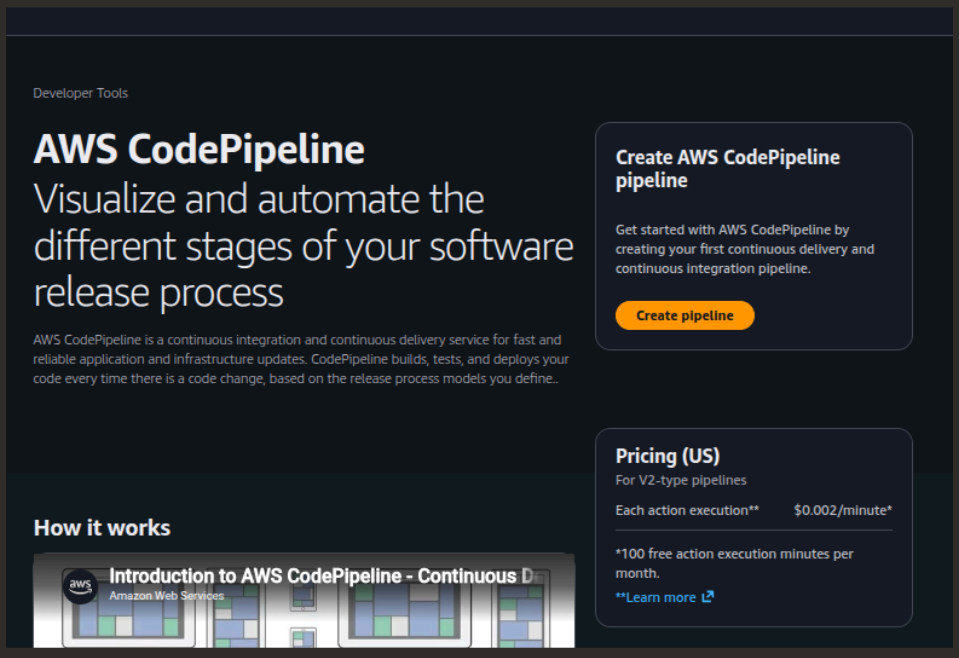
I chose to do this project today to learn how to build a pipeline using AWS CodePipeline and to understand its importance in DevOps CI/CD operations.

Starting a CI/CD Pipeline

AWS CodePipeline is an AWS DevOps tool that automates the workflow for moving code through the release process. In my setup, it takes code from GitHub, my source code repository, and moves it through to CodeDeploy, my deployment tool. This automation helps ensure deployments are consistent and reliable, reducing the risk of human error along the way.

CodePipeline offers different execution modes based on how it handles multiple runs of the same pipeline. I chose Superseded, which means my latest pipeline run takes priority over older, unfinished runs. The other options are Queued, which processes every run in the order it was started, and Parallel, which allows multiple runs to execute at the same time.

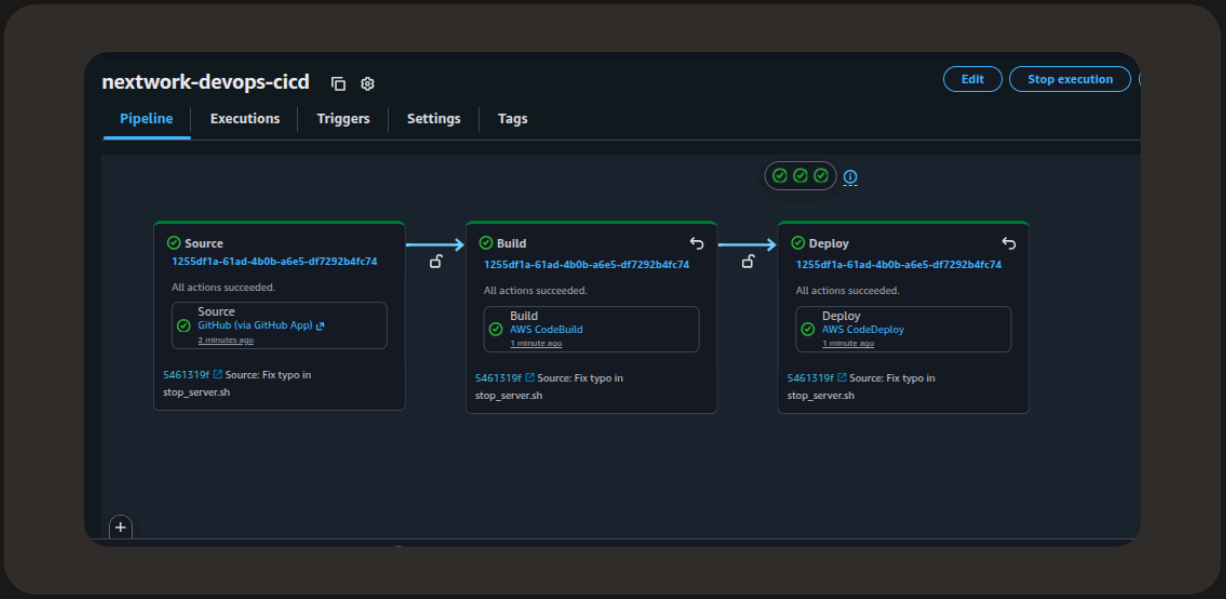
A service role gets created automatically during setup, giving CodePipeline the access it needs to coordinate each stage: connecting to GitHub through CodeConnections, and using CodeBuild and S3 to build and store artifacts.



CI/CD Stages

The three stages I set up in my CI/CD pipeline are Source (the web project's source code hosted in GitHub), Build (building the web app using CodeBuild), and Deploy (deploying the changes using CodeDeploy). While setting up each part, I learnt how each stage functions within a CI/CD pipeline.

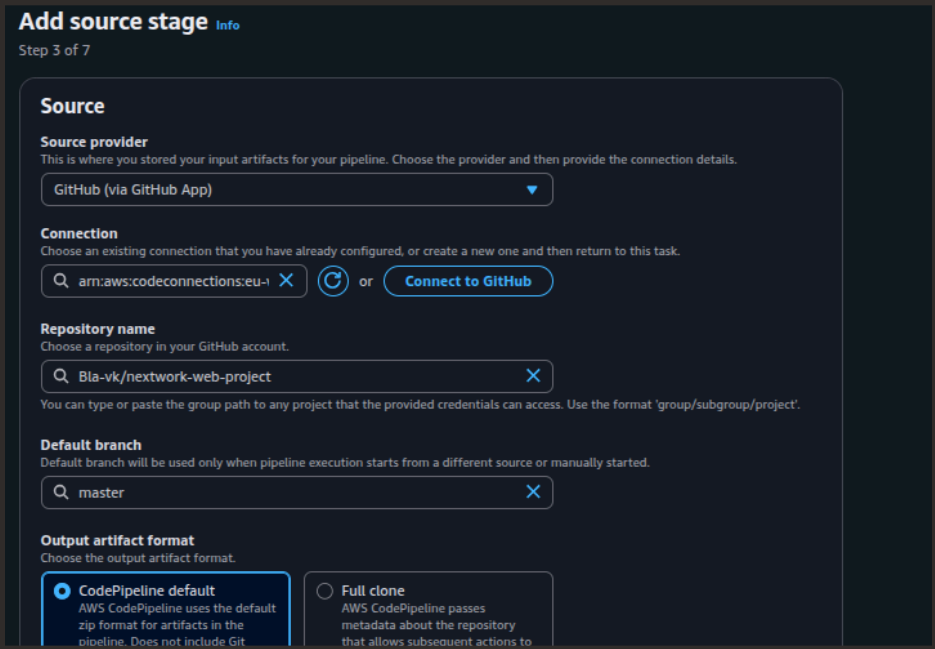
CodePipeline organizes the stages into a single diagram that shows the flow from Source to Deploy. In each stage, you can see details on the pipeline execution it belongs to, along with shortcuts to the connected service.



Source Stage

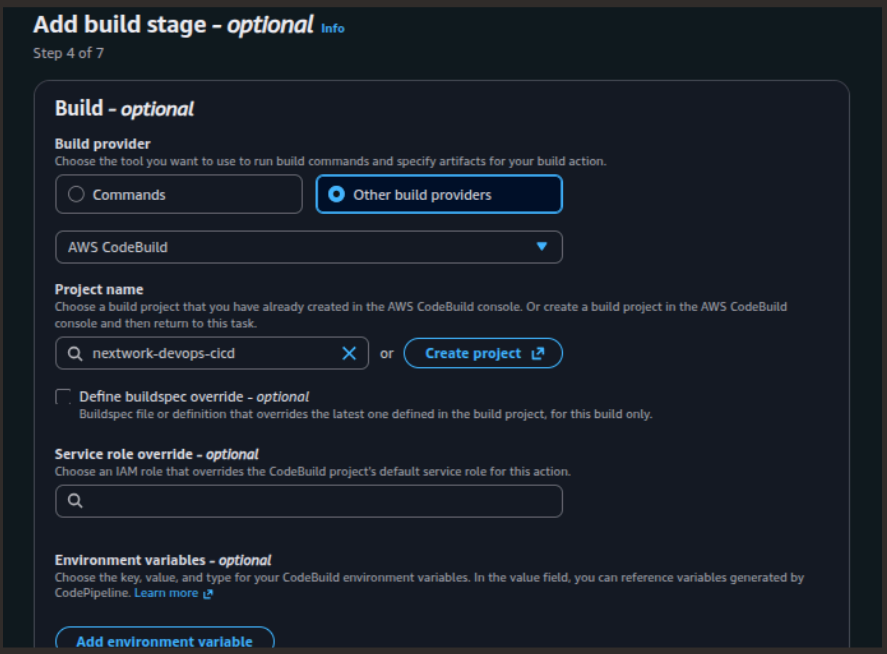
In the Source stage, which points to where the source code lives (GitHub), specifying the branch tells CodePipeline exactly which version of the code to use in this workflow. In production environments, you typically have several branches representing different versions or stages of your code or web app. By specifying a particular branch, you ensure that pipeline deploys that exact version, rather than picking up whichever branch happens to be active.

The source stage is also where you enable webhook events, which act like notifications. Whenever you make a change to the source code, the webhook event detects the change and alerts CodePipeline to trigger a new run.



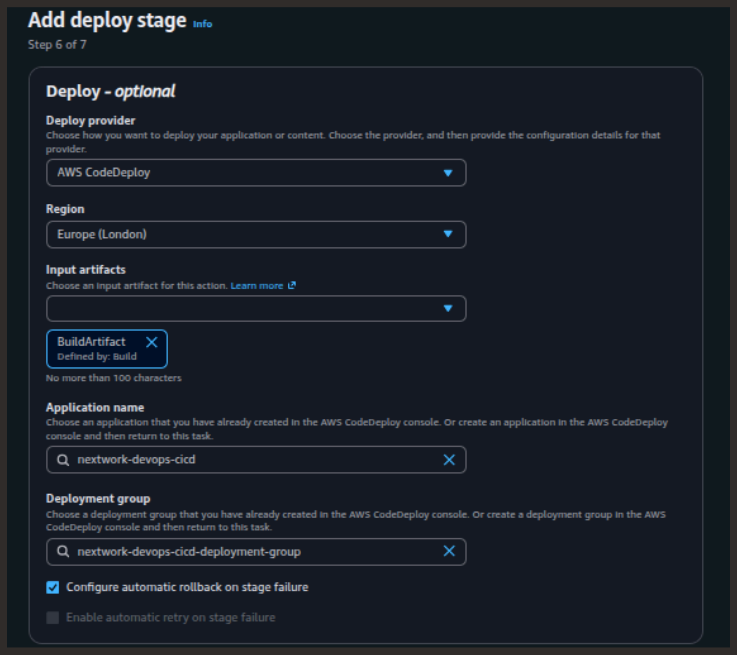
Build Stage

The Build stage sets up how my web app will be built and prepared for deployment. I configured CodeBuild as my build provider and set it to use the input artifact produced by the Source stage. That input artifact is the compressed source code, which the Source stage retrieved from GitHub and packaged into a zip file.



Deploy Stage

The Deploy stage is where I configure CodeDeploy as my deployment provider. It takes the build artifact from CodeBuild, along with the application and deployment group settings I've defined, to deploy the application.

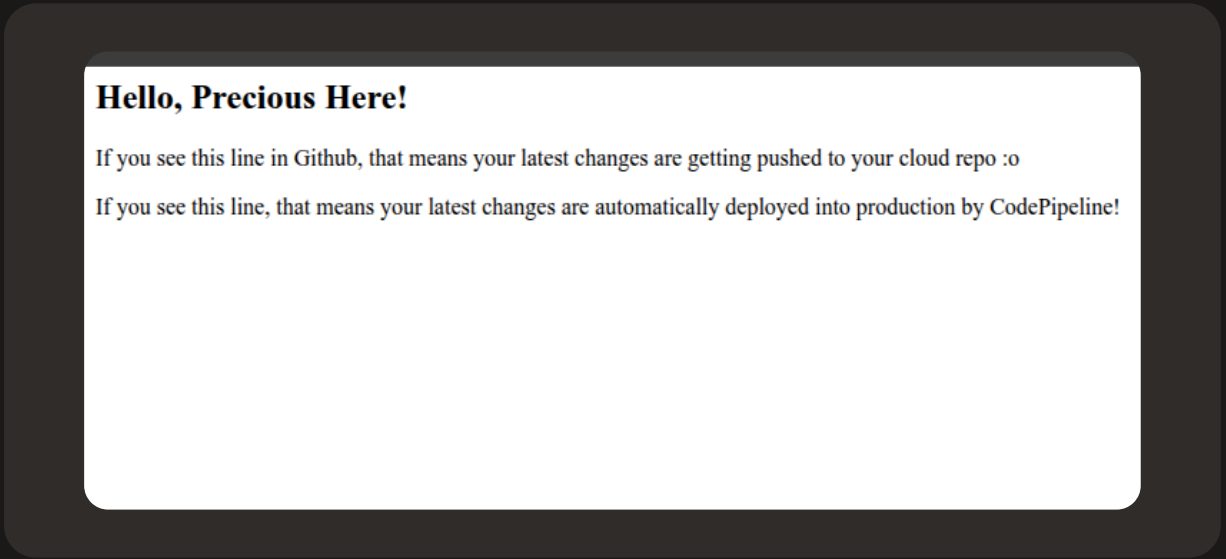


Success!

Since my CI/CD pipeline is triggered by code changes and webhook events, I tested it by updating the code. I added a line to my web app's index.jsp file and pushed the changes to trigger the pipeline.

The moment I pushed the code change, CodePipeline responded immediately, triggering a new pipeline execution through the Build and Deploy stages. The commit message under each stage reflects the latest code change being used. The Source stage was the first to reflect my latest commit message, and eventually the Build and Deploy stages showed the same thing too.

Once my pipeline executed successfully, I checked my live web app and confirmed it had updated automatically, without me needing to manually rebuild the project in CodeBuild or redeploy it through CodeDeploy. The changes went live into production immediately.



Testing the Pipeline

In a project extension, I initiated a rollback on the Deploy stage. Automatic rollback matters in situations where a deployed change passes automated tests but still turns out to be failing in practice, or doesn't behave the way it's supposed to once it's live.

During the rollback, the source and build stages were unaffected because both stages occur before the deployment stage. I verified this by comparing the commit messages associated with each stage: the deploy stage's commit message reverted to the commit from my 2nd pipeline run, while the source and build stages still reflect my latest commit.

After the rollback, the live web app reverted to its original state before I made any test changes. This confirms that CodeDeploy rolled back successfully.

